Implementation Plan: Knowledge Assistant & Flashcards

Implements the *Knowledge Assistant & Flashcards: Requirements* PDF inside the existing **KnowledgeReview** app, in phases. **Phase 1 = flashcard setup (generation pipeline). Phase 2 = Spectrum chat setup.** Later phases finish the rest of the PDF.

Related files: `PLAN-postgres-schema.md` (target tables), `PLAN-postgres-jwt.md` (done: Docker Postgres, JWT + refresh), `PLAN-spectrum-chat.md` (earlier chat design, superseded by phase 2 here).

Legend: **S** = a few days, **M** = about a week, **L** = more than a week (relative sizes, not dates; the PDF says estimate after phase 0).

---

0. Decisions already taken

Topic	Decision
App	Stay on the current **Next.js 16 + TypeScript** app (not the PDF's FastAPI + React/Vite). The PDF's features are built inside it
Database	**Postgres in Docker** (local and EC2). PGlite removed. **Done**
Login	JWT access token (15 min) + rotating refresh token, HttpOnly cookies. **Done**
Source of truth for cards	The **YAML in `kb/`** now (399 forms, 23 user controls, 4,927 rules, 408 concepts, 77 processes). The **knowledge graph later**, through a swappable "source provider"
Card generation	**Claude**, not templates. Output is validated and goes to Draft for admin approval
Claude access	**Claude subscription via the Claude Code CLI in headless mode** (`claude -p`), not the Agent SDK with an API key. See section 1 for the risk
Git	Branch `feature/spectrum-chat` in the KnowledgeReview repo, pull requests into `main`

1. Claude access: subscription, and what it means

The plan runs Claude by launching the **Claude Code CLI** (`claude -p ...`) from a worker process on EC2, logged in once with the owner's **subscription** account. Everything that touches Claude goes through one small interface so the credentials can change later without touching the rest of the app:

```
// lib/claude/runner.ts
interface ClaudeRunner {
  run(req: { prompt: string; system?: string; sessionId?: string; role?: Role;
             allowedTools: string[]; timeoutMs: number; maxTurns: number;
             onText?: (delta: string) => void; signal?: AbortSignal }): Promise<RunResult>; // text,
sessionId, tokens, durationMs, toolCalls, status
}
// implementations: CliRunner (subscription, now)  |  ApiRunner (API key or Bedrock, later)  |
MockRunner (tests)
```

Selected by `CLAUDE_RUNNER=cli|api|mock` .

Risks to accept or resolve in Phase 0 (they come straight from the PDF's risk section):

1. **Terms of use.** A personal subscription login serving several employees may not be permitted, and Anthropic's docs ask products built on the SDK to use API-key authentication. I cannot confirm what is allowed for an internal tool. **Phase 0 must get a written answer** from Anthropic or the account owner before Phase 2 goes live. If the answer is no, switch to `ApiRunner`: one setting plus an API key.
2. **Shared usage limits.** Subscription limits (rolling windows and weekly caps) are shared by every chat user and by the weekly card job. So the design must **pause and resume** when the limit is hit (section 2.4), cap concurrency at 3, and run the weekly batch at night in small resumable chunks.
3. **Login can expire.** The subscription session on the server can lapse. A health check runs a tiny `claude -p` every few minutes and alerts the admin when the login needs renewing.
4. **Credentials on the server.** `claude login` is done once, by SSH, as a dedicated OS user (`claude-runner`). Its credential files are readable only by that user. The web app's user cannot read them (REQ-NFR-05).

2. Shared building blocks (built in Phase 0 and 1, reused by Phase 2)

2.1 Harness directory

A dedicated folder, e.g. `/opt/spectrum/claude-harness/`, used as the working directory of every Claude run. It contains only:

```
.claude/settings.json            # permissions: allow list only, no hooks
.claude/skills/technical/SKILL.md
.claude/skills/functional/SKILL.md
.claude/skills/product/SKILL.md
.claude/skills/generate-cards/SKILL.md      # phase 1
.mcp.json                        # spectrum-kg MCP (phase 2). Absent for card generation
```

No source code, no `.env`, no secrets.

2.2 Tool permissions (REQ-HARN-01)

Enforced by CLI flags and settings, never by the prompt: allow only the KB MCP tools (`mcp__spectrum-kg__*`) plus web search and fetch for chat; **card generation allows no tools at all**. Deny Bash, Write, Edit, NotebookEdit, Read, Glob, Grep and sub-agents. Every denied attempt is written to the audit log. Exact flag names are confirmed against the installed CLI version in Phase 0 (`claude --help`).

2.3 Run shape

`claude -p "<prompt>" --output-format stream-json --include-partial-messages --max-turns N --model <model> --allowedTools ... --mcp-config ... --strict-mcp-config --append-system-prompt "<fixed system instruction>"`, with `cwd` set to the harness directory and `HOME` set to the `claude-runner` user. Follow-ups add `--resume <claude_session_id>`. The runner parses the JSON stream into text, tokens, tool calls and status, kills the process on timeout or Stop, and returns a typed result.

2.4 Limit handling

If the CLI reports a usage-limit or rate-limit error: mark the job `waiting_limit` (not failed), pause the queue until the reset time (or retry with backoff if unknown), show users "usage limit reached, your question is queued", and alert the admin. The weekly batch uses the same pause and resumes where it stopped.

2.5 Concurrency

One process-wide limiter (`MAX_CONCURRENT_CLAUDE`, default **3**, REQ-HARN-07), shared by chat and generation. Chat has priority over generation.

PHASE 0: Decisions and proof of concept (S)

Goal: remove the blockers before building.

Tasks

1. Get the **terms-of-use answer** for a subscription login serving an internal multi-user tool (section 1). Record it in the repo (`docs/decisions.md`).
2. Answer the open decisions: reviewer role (keep or merge into admin), self-signup (remove), spaced repetition (keep as optional or drop), `front/back` vs `title/body` naming, whether user controls get cards, what to do with the 818 template cards.
3. **Proof of concept on the EC2 instance** (as `claude-runner`):
   - `claude login` with the subscription; `claude -p "how many nodes are in the graph?"` answers through the spectrum-kg MCP.
   - A Bash attempt is denied and the denial is visible in the output.
   - Measure time to first text, total time, and RAM per Claude process; try 3 at once.
   - Check streaming output format, `--resume`, JSON structure for tokens and tool calls, and how a usage-limit error looks.
   - Check how a skill is applied in headless mode (as a slash command in the prompt, or by passing the skill text with `--append-system-prompt`).
4. Ask the KG team whether the MCP server accepts a user id and role per call (REQ-HARN-05).
5. Repo hygiene: commit the Postgres + JWT work on `feature/spectrum-chat`; CI running `typecheck`, `npm test`, `e2e` against a Postgres service container; install Playwright browsers.

Done when: one KB question is answered end to end through the CLI, a denied tool attempt is logged, the numbers (time, RAM, limit behavior) are written down, and the licence answer is recorded.

PHASE 1: Flashcard setup (generation pipeline) (L)

Goal: Claude generates form-wise, three-level cards from the `kb/` YAML, validated, versioned and stored in Postgres as Draft, with admin run controls and a review and publish page. (PDF section 7; REQ-GEN-01 to 10.)

1.1 Database (migration `0003_flashcard_generation`)

- `forms`: `id`, `code` (unique, e.g. fGRN), `name`, `module`, `kb_node_id` (`form:fGRN` now, the graph id later), `kind`, `active`, `synced_at`.
- `cards`: add `form_id` (nullable for legacy cards), `level` (1 Easy, 2 Medium, 3 Hard), `topic_key`, `is_control`, `control_of_card_id`, `altered_fact`, `flag_count default 0`. Unique (`form_id`, `level`, `topic_key`) where `form_id` is not null.

- `card_versions`: add `word_count`, `kb_node_ids text[]` (YAML references now), `batch_id`, `status` (draft / published / disputed / retired), `model`, `prompt_version`, `bundle_hash`. Statement cards use `front` as the title and `back` as the body (rename to `title` / `body` if decided in Phase 0).
- `batch_runs`: `id`, `trigger` (schedule/manual), `scope` (all / form code), `started_at`, `ended_at`, `status`, counts (created / changed / retired / rejected / unchanged), `tokens_in/out`, `error`.
- `batch_run_forms`: `batch_id`, `form_id`, `status` (pending / running / done / unchanged / failed), `bundle_hash`, `cards_created`, `cards_rejected`, `tokens`, `error`, `finished_at`. This is what makes **resume** possible (REQ-GEN-10).
- `settings` keys: `gen.cards_per_level` (default `{1:5, 2:8, 3:10}`), `gen.auto_publish` (default false), `gen.schedule` (default Sunday 02:00), `gen.parallel` (default 2), `gen.control_ratio` (default 0.1).
- Existing 818 template cards: mark `source = 'template'` and keep as is until the decision in Phase 0.

1.2 YAML source provider and source bundle (`lib/generation/`)

```
lib/generation/source.ts        # interface SourceProvider { listForms(); bundleFor(formCode): Bundle }
lib/generation/yaml-source.ts   # YamlSource: reads kb/ via the existing lib/cards/load.ts
lib/generation/bundle.ts        # builds + hashes the bundle
lib/generation/kg-source.ts     # placeholder for phase 7
```

Bundle contents (built by plain code, no AI), each item with a stable reference id:

- `form:<code>`: title, module, kind, users, summary, purpose, functional_description, actions, validations, method_notes.
- `rule:<key>` for every rule whose `applies_to` includes the form, skipping code-level rules (`isTechnicalRule`): name, type, summary, messages, and implemented stored procedures. **Rule descriptions are not included in full** (one is 116k characters); only a short trimmed description for the rules that Hard cards need.
- `concept:<key>` for concepts the form manages or uses: name, summary, tables.
- `process:<key>` for each process the form is a step of: name, summary, and the forms **before and after** it in the steps (needed for Medium cards, e.g. PO, then GRN, then Invoice).
- Size cap per bundle (about 12k tokens). Over the cap: keep rules by priority (calculation, derivation, integrity, validation, control, workflow, default) and record that it was truncated.
- `bundle_hash` = sha256 of the normalized bundle. Measured size on the current KB: median about 1.7k tokens, largest about 9.4k, all 399 forms about 0.9M tokens.

1.3 Generation skill and output contract

- `generate-cards/SKILL.md` (versioned as `prompt_version`): use **only** the supplied bundle; one **statement to verify** per card, not a question; level definitions and limits from the PDF (**Easy** 3-4 lines up to about 60 words: purpose, users, place in the flow; **Medium** 6-10 lines, 100-180 words: key fields, statuses, approval steps, forms before and after; **Hard** 10-20 lines, 200-350 words: validations, calculations, rules, posting and stock effects, tables and stored procedures, edge cases); cite the reference ids used; reuse the supplied topic keys for unchanged topics.
- Output is JSON validated by **zod**: `{ cards: [{ level, topic_key, title, body, refs[] }] }`. Checks: required fields, slug format and uniqueness of `topic_key` per level, word and line limits per level, requested number of cards per level, form exists, every `refs` entry exists in the bundle.

1.4 Grounding checks (anti-hallucination)

After the schema check, a program verifies every **number, quoted message, field name and screen name** in the card text appears in the bundle. Cards that fail are saved with `needs_review = true` and shown highlighted in the review page. Invalid cards are rejected and logged with the reason (REQ-GEN-05). One **repair call** per form re-asks only for the rejected cards, including the error text.

1.5 Versioning and storing (`lib/generation/persist.ts`)

Per form, in one transaction:

1. If `bundle_hash` equals the last successful run for that form, **skip** (status `unchanged`). Claude's wording varies between calls, so unchanged input must not be regenerated, otherwise every run would create false versions.
2. Otherwise generate. For each returned card, identity = form + level + topic key. Same content hash keeps the card; different content creates version N+1; a topic no longer returned retires its card (REQ-GEN-06).
3. New versions start as **Draft**. If `gen.auto_publish` is on, they publish immediately. **The previously published version stays visible until the new one is published**, then it is retired.
4. Every response points at the exact version shown (kept by the existing `card_version_id` links).

1.6 Control cards (REQ-GEN-08)

A second small call per form takes about 10% of the validated cards (at least one per level where possible) and returns a copy with **one fact deliberately changed**, plus the altered fact. Checks: the body differs, the change contradicts the bundle, same length limits. Stored with `is_control = true`, `control_of_card_id`, `altered_fact`. They look identical to normal cards and never appear in the corrections queue.

1.7 Batch orchestration (`scripts/generate-cards.ts`)

- Command: `npm run cards:generate -- [--form fGRN] [--all] [--resume <batch_id>] [--dry-run]`.
- Creates a `batch_runs` row and a `batch_run_forms` row per form, then works through forms with `gen.parallel` at a time, through the shared limiter (section 2.4).
- On a Claude usage limit, pause and resume (section 2.4). A crashed run is resumed with `--resume`, processing only forms that did not finish.
- Logs tokens and errors per form. Failing forms never block the rest.
- Schedule: a systemd timer (or cron) on EC2 calls the command weekly (default Sunday 02:00). An admin button runs it by hand.

1.8 Admin pages

- **Generation** (`/admin/generation`): "Run all" / "Run one form", progress of the current run, run history (start, end, forms processed, cards created / changed / retired / rejected, tokens, errors), schedule and the settings above.
- **Draft review** (`/admin/generation/review`): drafts grouped by form and level, each card shown beside the YAML items it cites, with needs-review warnings. Actions: publish, edit then publish, reject, publish all for a form or a batch.
- Add the card **level** to the existing browse screens, plus a level filter.

1.9 Existing template cards

Keep them published until the first approved Claude batch is live, then retire them (or keep as a separate "legacy" source). Decision taken in Phase 0.

1.10 Tests

- Unit: bundle builder is deterministic (same YAML gives same hash; changed rule changes the hash); zod limits per level; grounding check catches an invented number; version transitions (same / changed / retired / new); an unchanged re-run creates **zero** versions; resume processes only unfinished forms.
- Integration with `MockRunner` against the Docker Postgres test database.
- Pilot with the real CLI on **3 forms** (a small master, a transaction like GRN, and a big one like Purchase Bill).

Acceptance criteria

- A run on 3 forms produces valid cards at all three levels with sources cited, all saved as Draft.
- A second run with no YAML change creates **no** new versions and makes no Claude calls for those forms.
- Changing one rule in one form changes only the cards of that form.
- A run interrupted halfway resumes and finishes without redoing finished forms.
- An admin can review and publish; published cards appear for users.
- Run history shows counts and tokens.

Dependencies: Phase 0. **Main risk:** subscription limits during the first full run (about 2.4M output tokens for 399 forms). Mitigation: run in chunks over several nights, `gen.parallel = 1 or 2`, pause and resume.

PHASE 2: Spectrum chat setup (L)

Goal: employees ask questions about the knowledge base through a chat page; answers stream in, styled by the user's role; one login serves both the cards and the chat. (PDF sections 4, 5, 6; REQ-ROLE, AUTH, CHAT, HARN.)

2.1 Users, roles and audit (migration `0004_users_roles_audit`)

- `users`: add `user_type` ('admin' | 'general'), `answer_role` ('technical' | 'functional' | 'product', null for admin), `auth_source` ('local' | 'ess'), `must_change_password`, `failed_logins`, `locked_until`, `deactivated_at`. Map old roles: admin to admin; reviewer and employee to general with an answer role set by the admin.
- Admin-created accounts, password reset with forced change at next login, **no self-signup** (remove the signup and pending screens or hide them behind a flag). Lockout after **5 failures for 15 minutes**. Deactivating a user deletes their sessions (already true) and keeps their history.
- One append-only `audit_log` (time UTC, user, IP, event type, JSON payload, request id, session id). Merge `interaction_events` into it. Revoke UPDATE and DELETE on it from the app's database role. Every login attempt, tool call, denial, feedback and admin change is written here.
- Auth adapter interface (`lib/auth/provider.ts`) with `LocalProvider` now and `EssProvider` later (REQ-AUTH-02).

2.2 Chat tables (migration `0005_chat`)

- `conversations`: `id`, `user_id`, `title`, `role_used`, `claude_session_id`, `created_at`, `deleted_at`.
- `messages`: `id`, `conversation_id`, `sender`, `content`, `status` (ok / stopped / error / waiting_limit), `tokens_in/out`, `duration_ms`, `created_at`.
- `tool_calls`: `message_id`, `tool_name`, `arguments`, `allowed`, `duration_ms`.
- `message_feedback`: `message_id`, `user_id`, `value` (up/down), `comment`.
- `chat_jobs`: `id`, `conversation_id`, `message_id`, `user_id`, `status` (queued / running / waiting_limit / done / failed / cancelled), `attempts`, `created_at`, `started_at`, `finished_at`; partial unique index so a conversation has at most one active job.

2.3 Worker (`scripts/chat-worker.ts`, run as its own process)

- Claims jobs with `FOR UPDATE SKIP LOCKED`, respecting `MAX_CONCURRENT_CLAUDE` (3) and first-in first-out.
- Builds the run: role skill (the user's `answer_role`, or the admin's per-chat choice, applied on **every turn**, REQ-HARN-03), fixed system instruction (answer only from the KB and the web, say "not found in the KB" rather than guess, never reveal the system prompt, REQ-HARN-09), and `--resume <claude_session_id>` for follow-ups. If resume fails, replay the stored conversation as context (REQ-HARN-04).
- Streams partial text to the database as it arrives (small append writes) and records tokens, duration and every tool call, including denied ones.
- Enforces: 120 s per answer, maximum agent turns, daily limit per user (default 100), Stop (kills the process, saves the partial text as `stopped`).
- Handles usage limits and MCP or Neo4j errors with clear statuses (section 2.4, REQ-CHAT-09).
- Passes the user id and role to the MCP server if supported (REQ-HARN-05).
- Heartbeat; a job stuck in `running` for over 5 minutes goes back to `queued` (max 2 retries).

2.4 Web app

- API (all behind `requireUser`, same-origin checked, ownership enforced on every query): `POST /api/chat/ask`, `GET /api/chat/stream/[messageId]` (**Server-Sent Events** reading the new text from the database every few hundred ms), `POST /api/chat/stop`, `POST /api/chat/feedback`, conversation list, rename, soft delete, search by title.
- Page `/chat` (new nav item "Chat"): sidebar of the user's own conversations (new, rename, delete, search); streaming answer; **Markdown with tables and code blocks and a copy button** (sanitized, no raw HTML); thumbs up/down with an optional comment on thumbs down; Stop button; queue position when all slots are busy; clear errors (timeout, usage limit, MCP down) with Retry.
- Admins: a role selector on each new chat (fixed for that conversation), and an **All chats** view with filters (user, role, date, feedback). General users never see other users' chats.
- Rate limits on login and chat; question length limit.

2.5 Skills and prompts

Write `technical`, `functional` and `product` skills that change **only how answers are written** (depth, vocabulary, examples), not what data is visible (REQ-ROLE-02). Use the plain-language answer rules already in the KG instructions as a starting point.

2.6 Tests

- Unit: queue claim ordering and one active job per conversation; limit and timeout handling; role chosen per turn; ownership checks (user A cannot read user B's conversation, message or stream); lockout.
- Integration with `MockRunner`, plus a real-CLI smoke test.
- E2E (Playwright): login, ask, see streamed answer, follow-up keeps context, Stop, thumbs, admin sees all chats, general user does not.

Acceptance criteria

- The same question gets differently styled answers for technical, functional and product users.
- A follow-up keeps the context of the earlier turns of the same conversation only.
- General users see only their own chats; admins see all.
- With more than 3 questions at once, extra users see their queue position, and they are answered in order.
- Timeout, usage-limit and MCP-down errors are shown, and Retry works.
- Denied tool attempts (for example Bash) appear in the audit log; no approval prompt ever appears.

- One login works for both the cards pages and `/chat`; logout ends both.

Dependencies: Phase 0 (licence, PoC), shared runner from section 2. **Main risks:** subscription limits shared with the weekly batch (chat gets priority), expiring login (health check), memory (cap at 3 concurrent, t4g.large).

PHASE 3: Flashcard player and timing (M)

Goal: users study the published cards by form and level, with trustworthy timing. (PDF section 8.)

- Migration `0006_player`: `card_views` (opened_at server clock, answered_at, hidden_ms, idle_ms, raw_ms, active_ms capped at 10 minutes, outcome answered / abandoned) and `card_responses` (answer known / new / incorrect, reason, correction text of at least 15 characters, too_fast). Unique (`user_id`, `card_version_id`) so each version is answered once. Backfill old `reviews`.
- Player: form (search + list) and level filters, unanswered cards first then random, progress bar, "All done", **My answers** (read-only), keys 1, 2, 3, works on phone.
- Timing: the server records open and answer; the browser only reports hidden periods and idle periods over 60 s, which are **subtracted**, never added (REQ-SCORE-06). Closing, logging out or timing out records **abandoned**.
- Decide spaced repetition and daily decks: retire them, or keep as an optional "review mode".
- **Acceptance:** active time excludes hidden-tab time; one scored answer per card version; a card closed without answering is stored as abandoned.

PHASE 4: Corrections, scoring and leaderboard (L)

Goal: errors in the knowledge base are found and fixed; points reward careful reading. (PDF sections 9, 10.)

- Migration `0007_corrections_scoring`: `card_flags`, `flag_reviews`, `kb_fix_items`, `scoring_rule_sets`, `points_ledger` (append-only, with rule version).
- Corrections: a card with 2 flags (configurable) becomes **Disputed** and is hidden from users who have not answered it; admin review queue with all flags side by side and the cited YAML items; decisions confirmed / rejected / partial with notes; confirmed cards retire and add a KB fix item; fix list exportable as CSV; users see the outcome of their own flags. Control cards are never queued: auto-confirm when the reason matches the altered fact, otherwise hold for an admin.
- Scoring (all values editable, defaults from the PDF): known and new earn the level base (Easy 1, Medium 2, Hard 3); too-fast answers earn 0; flag confirmed 5 x level base; control card caught 3 x; control card marked "known" -3 x; 200 points per day cap on known/new; first answer only; minimum read time = words / 5 per second, never under 4 seconds; five too-fast answers in a row pause the player for 2 minutes; about 10% control cards; reliability score visible to admins only.
- Leaderboard: global and per form, this week and all time, tie-break by confirmed corrections then who reached the score first; refreshed within 1 minute. Admins recalculate from the ledger after rule changes.
- **Acceptance:** a unit test for every row of the points table; a control card marked correct deducts points; a confirmed flag awards points to every flagger.

PHASE 5: Admin analytics and export (M)

- The metrics in PDF section 11 (answers by user / form / level / week, answer mix, average and median active time, too-fast rate, reliability, flag accuracy, form coverage grid, most disputed cards, abandoned cards, chat usage, batch runs), built as database views plus pages with charts where trends help.
- Per-user detail page with a timeline; CSV and Excel (.xlsx) export of every table with the active filters.
- **Acceptance:** exported numbers match the database; pages load in under 3 seconds with 15 users and a year of data.

PHASE 6: Hardening and go-live (M)

- **Security:** HTTPS only, ports 443 and 22 public only, Postgres and the MCP server reachable only inside the server; secrets in env files or AWS SSM, never in Git; the Claude user is non-root and cannot read app secrets; CSRF, input validation, security headers. **Prompt-injection tests** (try to run a shell command, read files, reveal the configuration): every attempt must fail.
- **Operations:** Docker Compose with `nginx`, `web` (Next.js), `chat-worker`, `generation` timer, `postgres`, `mcp-server`; daily Postgres dump kept 30 days and copied to S3 with a **tested restore**; monitoring (container health, disk above 80%, memory above 85%, failed batch run, **Claude login expired**); CI deploys; EC2 sized up to t4g.large.
- **Quality:** Vitest unit tests, Playwright e2e for login, chat, cards and scoring; a short user guide.
- Remove the old tables (`reviews`, `feedback`, `interaction_events`, and `srs_state` / `daily_decks` if retired) after their replacements are proven.
- **Acceptance:** prompt-injection tests all fail to break out; a backup is restored successfully; admins sign off.

PHASE 7: Switch generation to the knowledge graph (S to M, later)

- Implement `KgSource` returning the **same bundle shape** from the graph: form node, rules through APPLIES_TO, concepts through MANAGES / USES_CONCEPT, processes through HAS_STEP, and tables and stored procedures through USES_TABLE and EXECUTES for richer Hard cards.
- The reference ids already match the graph's id convention (`form:<Form>`, `rule:<key>`, `concept:<key>`, `process:<key>`), so stored `kb_node_ids` carry over.
- Validation, versioning, storage, review and corrections do not change. Switch with `GEN_SOURCE=kg`.
- Compare YAML-sourced and graph-sourced cards for a few forms before switching fully.

Traceability: PDF requirements to phases

PDF area	Requirements	Phase
Roles and permissions	REQ-ROLE-01..05	2
Authentication	REQ-AUTH-01..07	done (tokens), 2 (rest)
Chat	REQ-CHAT-01..12	2
Claude harness	REQ-HARN-01..10	0, 2 (and 1 for HARN-10)
Card generation	REQ-GEN-01..10	1

PDF area	Requirements	Phase
Player	REQ-CARD-01..08	3
Corrections	REQ-CORR-01..04	4
Scoring, leaderboard	REQ-SCORE-01..08, REQ-LB-01..05	4
Analytics	REQ-AN-01..04	5
Audit	REQ-AUD-01..04	2 (log), 5 (viewer)
Non-functional	REQ-NFR-01..13	0, 6

Configuration summary (new variables)

```
CLAUDE_RUNNER=cli            # cli | api | mock
CLAUDE_BIN=claude
CLAUDE_HARNESS_DIR=/opt/spectrum/claude-harness
CLAUDE_MODEL=
MAX_CONCURRENT_CLAUDE=3
CHAT_TIMEOUT_SECONDS=120
CHAT_MAX_TURNS=
CHAT_DAILY_LIMIT=100
GEN_SOURCE=yaml              # yaml | kg
GEN_PARALLEL=2
KB_DIR=./kb
KG_MCP_URL=                  # http://localhost:<port> on EC2
KG_MCP_TOKEN=
ANTHROPIC_API_KEY=           # only if CLAUDE_RUNNER=api
```

Order, dependencies and what to do first

```
Phase 0 (decisions, PoC, licence)
   ├─> Phase 1 (cards generation)  ──> Phase 3 (player) ──> Phase 4 (corrections, scoring) ──> Phase 5
(analytics)
   └─> Phase 2 (chat)  ──────────────────────────────────────────────────────────────────────┐
                                                                                             Phase 6 (hardening, go-live)
<┘
Phase 7 (knowledge graph source) any time after Phase 1 is stable
```

Phase 1 and Phase 2 can overlap after the shared runner exists, but Phase 1 goes first as requested. Start with Phase 0 items 3 and 1 (the proof of concept and the licence answer), because every later phase depends on them.

Open questions (carried from the PDF and this plan)

1. Licence answer for a subscription login on an internal multi-user app (blocks Phase 2 go-live).
2. Reviewer role: merge into admin, as the PDF has only admin and general?
3. Spaced repetition: retire or keep as an optional mode?
4. Do user controls (23) get cards, or only forms?

5. Template cards: retire after the first approved Claude batch?
6. Auto-publish or admin approval for each batch (default: approval)?
7. Cards per level (default 5 / 8 / 10); scoring values; answer reasons list.
8. Are admins on the leaderboard? Leaderboard periods: this week and all time enough?
9. Does the MCP server accept user id and role per call?
10. ESS login: how can we validate against it (API, LDAP, database) and who owns it?
11. Domain name and certificate for the app.